Verdict Runtime

BENCHMARK
2026-10-24
12 apps · 6 languages

We find more labelled flaws than Snyk Code and CodeQL

Head-to-head on twelve deliberately vulnerable applications, scored against each app's own labels, plus a live layer that *proves* findings.

On **12 deliberately vulnerable applications across 6 languages**, Verdict Runtime found **107 of 188** labelled, statically findable flaws. **Snyk Code found 81** and **GitHub CodeQL found 78**. On an industry-standard Python benchmark it scored **44.5** to Snyk Code's **27.5**.

107/188
Flaws found — Verdict Runtime

81
Snyk Code · +26 for us

78
GitHub CodeQL · +29 for us

44.5
Python benchmark score (Snyk Code 27.5)

Round 1: code scanning, like for like

A flaw counts as found only when the correct vulnerability class is reported within ±3 lines of the labelled location, scored against an answer key built from each app's own labels.

SCANNER	LABELLED FLAWS	FOUND	SHARE
Verdict Runtime	188	**107**	**57%**
Snyk Code	188	81	43%
GitHub CodeQL	188	78	41%

Round 1b: OWASP-style Python benchmark, 1,230 cases

63% of the cases are safe code made to look vulnerable. Score = share of real flaws found minus share of safe cases flagged (100 is perfect, 0 is chance); a hit counts only if it names the right weakness type (CWE).

CLASS	US	SNYK CODE
SQL injection	**100**	0
Weak hashing	100	100
Insecure cookie	100	100
Command injection	**69.2**	16.5
Code injection	**66.7**	13.6
Insecure deserialization	**66.7**	55.6
Open redirect	**56.8**	40.7

CLASS	US	SNYK CODE
Trust boundary	**44.4**	0
Path traversal	**39.1**	−3.2
LDAP injection	**37.5**	0
XPath injection	**36.5**	23.2
Cross-site scripting	**19.1**	18.0
XML external entities	0	0
Weak randomness	0	0
Overall	**44.5**	**27.5**

10 classes won, 4 tied, none lost. 54% of real flaws found vs 38%, with fewer safe cases flagged (9.5% vs 10.3%).

The layer the static scanners don’t have LIVE PROOF

9 additional vulnerabilities confirmed by actually exploiting three of the running apps

Verdict Runtime doesn’t just read code: its agents send real, scope-authorised requests and **prove** each finding against the live application, with a reproducing request each:

- **Broken object authorisation (BOLA/IDOR)** — one account read another account’s private data
- **Authentication-token forgery** — an unsigned (`alg:none`) token accepted as valid
- **Exposed admin / debug surfaces** — internal config and server logs served to an unauthenticated request
- **Missing login rate-limiting & weak session cookies**

Static scanners, Snyk Code and CodeQL included, cannot prove any of these. Broken authorisation and token forgery are logic flaws that only show up when you exercise the live application.

Method & scope. Twelve deliberately vulnerable training applications in six languages (JavaScript/TypeScript, Python, Java, Ruby, Go, C#), each scored against an answer key built from the app’s own labels; a detection counts only when the correct vulnerability class is reported within ±3 lines. Each competing scanner ran at its documented security settings (Snyk Code; CodeQL security-extended suites). The Python benchmark is synthetic and an early release; one run per tool (Verdict Runtime 2026-10-24, Snyk Code 2026-10-23). The live-proof results come from Verdict Runtime’s agents against disposable, explicitly authorised local instances. These are labelled training targets, so the results show a measured lead on this suite, not a claim of universal superiority. Snyk is a trademark of Snyk Limited; CodeQL is a trademark of GitHub, Inc.